

CURSO DE PYTHON

**Corporación Ecuatoriana para el Desarrollo de la
Investigación y la Academia**

Ing. María Fernanda Granda J., PhD

27-07-2026

**“Conectando ideas,
transformando
sociedades”**

Tema 6: Computación Científica con NumPy

Objetivo de Aprendizaje:

Al finalizar la sesión, el estudiante será capaz de utilizar la biblioteca NumPy para crear y manipular arreglos multidimensionales, realizar operaciones matemáticas y estadísticas de forma eficiente, optimizando el procesamiento de grandes volúmenes de datos para aplicaciones de ingeniería, ciencia de datos e inteligencia artificial.

Competencias:

- Comprender las ventajas de NumPy frente a las listas de Python.
- Crear y manipular arreglos unidimensionales y multidimensionales.
- Aplicar operaciones vectorizadas para mejorar el rendimiento.
- Resolver problemas de álgebra lineal utilizando NumPy.
- Calcular medidas estadísticas básicas sobre conjuntos de datos.

Tema 6: Computación Científica con NumPy

- Introducción a NumPy
- Creación de arreglos
- Operaciones vectorizadas
- Indexación y Manipulación
- Operaciones de Algebra Lineal
- Estadística descriptiva
- Actividades y Tarea

Recursos necesarios

- Python 3.11+
- Visual Studio Code
- Biblioteca NumPy

Instalación:

```
pip install numpy
```

```
python3 -m pip install numpy
```

¿Por qué utilizar NumPy?

- Revisemos estos dos programas que realizan la suma de un millón de números.

Opción 1

```
import time

inicio = time.time()

numeros = list(range(1000000))

resultado = [x * 2 for x in numeros]

fin = time.time()

print(fin - inicio)
```

Discusión

- ¿Cuál fue más rápido? ¿Por qué? ¿Qué ventajas tiene NumPy?

Opción 2

```
import numpy as np
import time

inicio = time.time()

numeros = np.arange(1000000)

resultado = numeros * 2

fin = time.time()

print(fin - inicio)
```

0.11842060089111328

0.009572029113769531

¿Qué es NumPy?

- NumPy significa Numerical Python.
 - Es la biblioteca principal en Python para el cálculo numérico, el análisis de datos y la computación científica.
 - Sus componentes clave son los arreglos multidimensionales (arrays), las operaciones vectorizadas y las funciones matemáticas eficientes.

Permite:

- Computación científica
- Procesamiento de datos
- Inteligencia Artificial
- Machine Learning
- Simulación
- Ingeniería.

Creación de Arreglos

Una lista es nativa y flexible para guardar varios tipos de datos

Un array guarda elementos de un solo tipo y es más rápido para matemáticas.

Comparar

- Tipo
- Operaciones
- Velocidad

```
# crear un arreglo
import numpy as np
import time

#Con numpy
print("Crear array con numpy")
inicio = time.time()
a = np.array([10, 20, 30, 40])
print(a)
fin = time.time()
print(fin - inicio)

#usando listas
print("Crear array con lista")
inicio = time.time()
lista = [10, 20, 30, 40]
array = np.array(lista)
print(array)
fin = time.time()
print(fin - inicio)
```

```
#-----
#Ceros
print("Generar 5 ceros")
print(np.zeros(5))

#-----
#Unos
print("Generar 4 1s")
print(np.ones(4))

#-----
#Rango
print("Crear un rango de números de 0 a 20 con incremento de 2")
print(np.arange(0, 20, 2))
```

```
#-----
#Espacios iguales
print("Crear espacios iguales entre 0 y 1")
print(np.linspace(0, 1, 5))

#-----
#Matriz Identidad
print("Genera una matriz identidad de 4x4")
print(np.eye(4))

#-----
# Crear una matriz de 2x3 con datos manuales
matriz = np.array([[1, 2, 3], [4, 5, 6]])
print(matriz)
print("Forma (filas, columnas):", matriz.shape)
```

Operaciones Vectorizadas

Operaciones Básicas:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a+b) #suma
print(a*b) #multiplicación
print(a/b) #división
print(a**2) #potencia
```

Resultado:

```
[5 7 9]
[ 4 10 18]
[0.25 0.4 0.5 ]
[1 4 9]
```

Operaciones Vectorizadas

Operaciones Matemáticas:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(f"Arreglo a: {a}")
print(f"Arreglo b: {b}")
print(f"Raíz cuadrada de a: {np.sqrt(a)}") # raíz cuadrada
print(f"Seno de a: {np.sin(a)}") # seno
print(f"Coseno de a: {np.cos(a)}") # coseno
print(f"Logaritmo a: {np.log(a)}") # Logaritmo
print(f"Comparar a>2: {a > 2}") # Comparaciones
print(f"Filtrado de a>2: {a[a > 2]}") #Filtrado
```

Resultado:

```
[1.          1.41421356  1.73205081]
[0.84147098  0.90929743  0.14112001]
[ 0.54030231 -0.41614684 -0.9899925 ]
[0.          0.69314718  1.09861229]
[False False  True]
[3]
```

Actividad 1

Dado:

```
ventas = np.array([120, 340, 560, 100, 90, 450])
```

Realizar:

- Incrementar todas las ventas un 10%.
- Mostrar ventas mayores a 300.
- Calcular el IVA del 15%.
- Obtener el cuadrado de cada venta.

Indexación y Manipulación de Arreglos

Matrices:

```
import numpy as np

matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"Matriz: {matriz}") # mostrar matriz
print(f"Elemento [0,1]: {matriz[0,1]}") #Acceder elementos
print(f"Fila [1]: {matriz[1]}") #Mostrar fila 1
print(f"Columna [2]: {matriz[:, 2]}") #Mostrar columna 2
print(f"Submatriz: {matriz[0:2, 1:3]}") #Mostrar submatriz
a = np.arange(12) # genera números
a = a.reshape(3,4) # ordena de 3 x 4
print(f"Cambiar forma: {a}") #Mostrar
```

Resultado:

```
2
[4 5 6]
[3 6 9]
[[2 3]
 [5 6]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Operaciones de Algebra Lineal

Operaciones con matrices:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B) # multiplicación de matrices
print(A.T) # transpuesta
print(np.linalg.det(A)) # Determinante
print(np.linalg.inv(A)) # inversa
```

Resultado:

```
[[19 22]
 [43 50]]
[[1 3]
 [2 4]]
-2.0000000000000004
[[-2.  1. ]
 [ 1.5 -0.5]]
```

Operaciones de Algebra Lineal

Resolver sistemas de Ecuaciones:

La matriz **A** y el vector **b** representan el siguiente sistema:

$$\begin{cases} 2x + y = 7 \\ x - y = 1 \end{cases}$$

Resultado:

```
A = np.array([[2, 1], [1, -1]])
b = np.array([7, 1])
print(A)
print(b)
print(np.linalg.solve(A, b)) # muestra las raíces
```

```
[[ 2  1]
 [ 1 -1]]
[7 1]
[2.66666667 1.66666667]
```

Estadística Descriptiva

Matrices:

```
import numpy as np
# ejemplo de estadística descriptiva
notas = np.array([8, 7, 9, 10, 6, 8, 7])
```

```
print(np.mean(notas))      # Media
print(np.median(notas))    # mediana
print(np.max(notas))       # máximo
print(np.min(notas))       # mínimo
print(np.std(notas))       # desviación estándar
print(np.var(notas))       # Varianza
```

Resultado:

```
7.857142857142857
8.0
10
6
1.2453996981544782
1.5510204081632655
```

Tarea

Análisis de Resultados Académicos

Genere una matriz con las calificaciones de 30 estudiantes en 5 asignaturas.

Deben desarrollar un programa que:

- Calcule el promedio por estudiante.
- Calcule el promedio por asignatura.
- Identifique la nota más alta y la más baja.
- Determine cuántos estudiantes aprobaron cada asignatura (nota ≥ 7).
- Muestre un ranking de estudiantes según su promedio.

GRACIAS